UK PLANNING INTELLIGENCE

Twelve million planning documents, made *searchable*

The UK holds more than 12 million planning documents, over 120 million pages in every format going, and a lot of them are scans. For a UK planning-intelligence company, I built the pipeline that turns all of it into clean text and embeddings. It's one Docker image scaled out on spot instances, and it did the full national backfill in under 48 hours, with self-hosted embeddings, so there's no per-token bill.

12M+ DOCUMENTS IN 48 HOURS	0 ERRORS IN 63,809-DOC RUN
	$0 PER-TOKEN EMBEDDING COST
	2.5M PAGES PER HOUR

ADAM MURPHY

AI ENGINEER · EXPERT-VETTED ON UPWORK

THE SITUATION

A national archive nobody could read

The UK's planning record is huge and messy. More than 12 million applications sit on council portals up and down the country (decisions, officer reports, design statements, site plans), which comes to over 120 million pages of PDFs of every vintage, Word files, spreadsheets and images, and a lot of it is scanned paper. No two councils publish the same way, so for a machine most of it might as well not exist.

My client, a UK planning-intelligence company, needed all of it readable. Their product, LandGPT, answers questions about land and development, and you can only trust an answer if it cites the actual document. So the brief was to extract the text from every planning document in the country and embed it for retrieval, at a cost that made national coverage viable.

WHAT I BUILT

One image that handles any format at any scale

The pipeline is a scheduler and worker system on SQS with a dead-letter queue, all packed into a single Docker image that can run as either role. That makes it easy to scale out, so for the national backfill it fanned out across Kubernetes spot instances and cleared 12 million documents in under 48 hours, which is roughly 2.5 million pages an hour. Day to day it runs quietly and takes in 1,000+ new applications every day.

01
Schedule
The scheduler queues each document on SQS and a dead-letter queue catches anything that keeps failing

→

02
Extract
Native parsers go first, then vision OCR for scans, then a second provider as fallback

→

03
Embed
Sentence and paragraph embeddings, self-hosted, so there's no per-token cost

→

04
Store & verify
Rerun-safe upserts to pgvector on TimescaleDB, and a reconciliation check proves nothing is missing

I set extraction up as a ladder. Checking a file's opening bytes tells me what it is, since extensions often lie, and then native parsers handle 10+ formats cheaply, with a five-stage encoding fallback for awkward text. A document only goes up to Gemini Flash vision OCR when nothing readable comes out, and only goes to an alternate provider if Gemini refuses. Every chunk then gets embedded twice, at sentence level and paragraph level, with a self-hosted nomic-embed-text model on Ollama, so retrieval can be precise or contextual without a per-token bill.

63,809	8h54m	10+	1,000+
DOCS, ONE PORTAL, ONE RUN	FOR THAT RUN, ZERO ERRORS	FILE FORMATS HANDLED	NEW APPLICATIONS DAILY

THE HARD PARTS

What breaks at 120 million pages

Every format going, most of them mislabelled

Council uploads arrive mislabelled, corrupted or as scans of typewritten pages. So the ladder trusts a file's actual bytes over its extension, tries the cheap native parser first and only uses vision OCR as a last resort, which is what keeps a 120-million-page corpus affordable.

Models that refuse public documents

The vision model would sometimes refuse to transcribe perfectly public planning documents because it flagged the verbatim text as recitation. I didn't want to lose those files, so the pipeline falls back to an alternate provider's model and keeps prompting it to carry on until a completion marker shows up, which stitches long documents together.

Embedding costs that scale with the country

At 64,000 documents from a single portal, per-token embedding APIs price you out. Self-hosting nomic-embed-text on Ollama took the cost of each extra embedding to zero, so having both sentence and paragraph embeddings came free instead of doubling the bill.

Workers die mid-document all the time

On spot instances, workers die halfway through a document as a matter of course. I keyed every write on document, chunk and level, so rerunning anything costs nothing, and retries plus the dead-letter queue soak up the rest. The reconciliation pass at the end of each run is there to prove nothing was missed.

THE RESULTS

BACKFILL	**12M+** documents (120M+ pages) in under 48 hours on K8s spot instances, or about **2.5M** pages an hour
RELIABILITY	**63,809** documents from one council portal in 8h54m with **zero errors**, and reconciliation confirmed 0 left unprocessed
COST	**$0** per-token embedding cost on self-hosted models, with an LLM used only as an OCR fallback
PRODUCT	Powers LandGPT's document citations, and the client measured chatbot engagement up **22%** after citations shipped. 1,000+ new applications come in daily

A NOTE ON CONFIDENTIALITY

I've kept the client anonymous (a UK planning-intelligence company), though the product name, LandGPT, is public. The portal-run figures of 63,809 documents, 8h54m and zero errors come straight from production run logs. The 12-million-in-48-hours figure comes from the initial national backfill on spot instances, which was the pipeline's largest single run. This work is part of an eighteen-month relationship with the same client, which started with a 5.0★ Upwork contract over 758 hours.

ABOUT ADAM

I'm a freelance Senior AI Engineer, **Expert-Vetted on Upwork (top 1%)**, with 100% Job Success across 75+ projects, $500K+ earned and 6,000+ hours over seven years. I build production LLM systems for regulated industries like insurance, finance and law.

Upwork profile ↗ · github.com/codeananda ↗